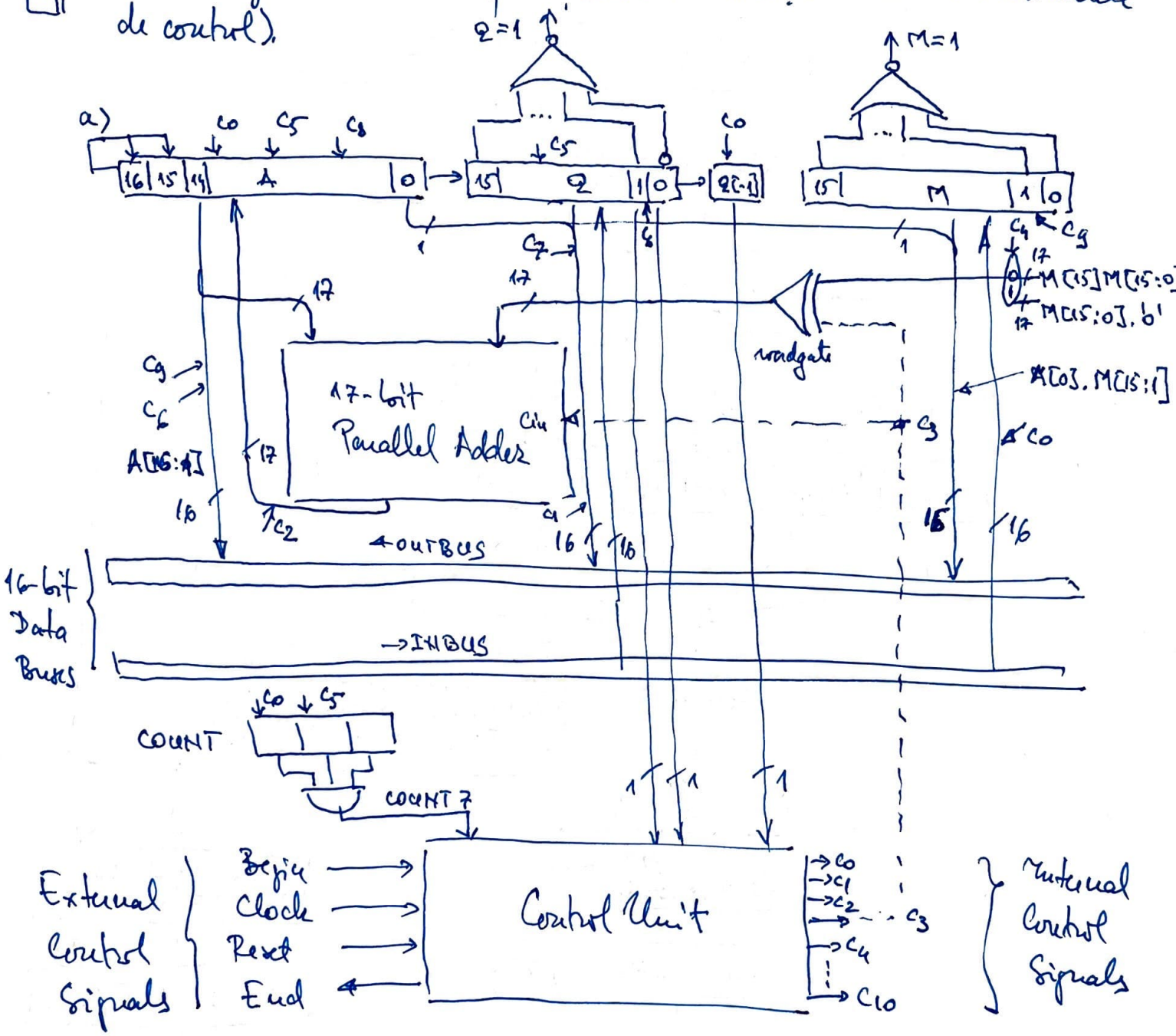
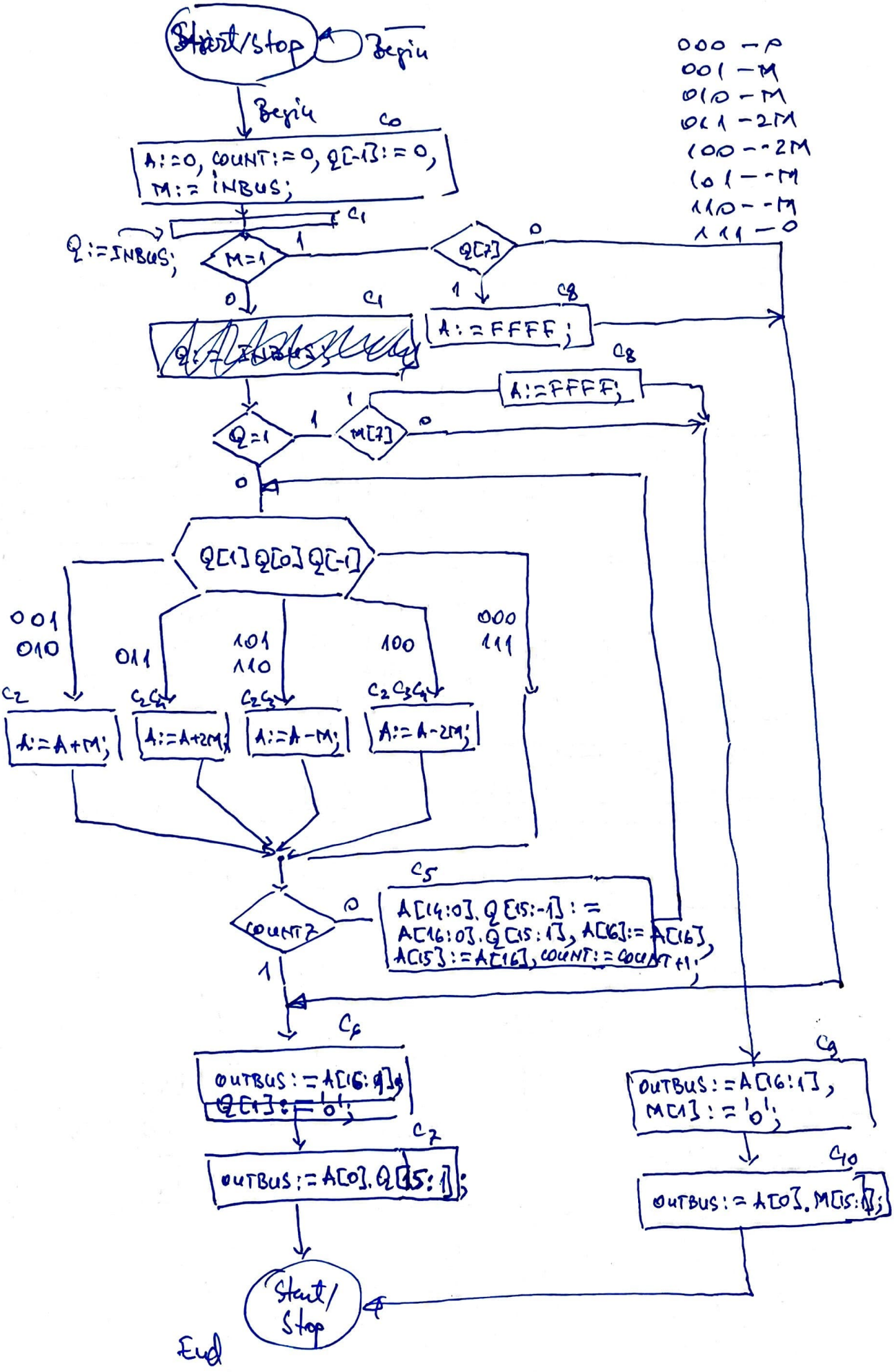
(2p) (1) Să se modifice dispozitivul de înmulțire Booth radix-4, astfel încât acesta să înmulțească numere fracționare reprez. pe 16 biți și să trateze în mod distinct situațiile de înmulțire cu unu. (Rezultatul este egal cu celălalt operand, exprimat pe 32 biți.) Se cer:

[1p] a) Schema bloc detaliată pe platforma ~~[illegible]~~ Hayes. (cu semnale de ctrl.)

[1p] b) Ordinograma ce descrie funcționarea unității de control (cu semnalele de control).




Start/stop
Begin
Begin
c0
A:=0, COUNT:=0, Q[-1]:=0,
M:=INBUS;
c1
Q:=INBUS;
M=1
1
0
Q[7]
0
1
c1
c8
A:=FFFF;
A:=FFFF;
c8
Q=1
1
M[7]
1
0
0
Q[1] Q[0] Q[-1]
001
010
011
101
110
100
000
111
c2
c2c4
c2c3
c2c3c4
A:=A+M;
A:=A+2M;
A:=A-M;
A:=A-2M;
c5
A[14:0], Q[15:-1]:=
A[16:0], Q[15:1], A[6]:=A[16],
A[5]:=A[16], COUNT:=COUNT+1;
COUNT7
0
1
c6
OUTBUS:=A[16:9];
Q[1]:=1'b0;
c7
OUTBUS:=A[8:1];
c9
OUTBUS:=A[16:1],
M[1]:='0';
c10
OUTBUS:=A[0], M[15:1];
Start/
Stop
End
000 - P
001 - M
010 - M
011 - 2M
100 - -2M
101 - -M
110 - -M
111 - 0

A1
B2

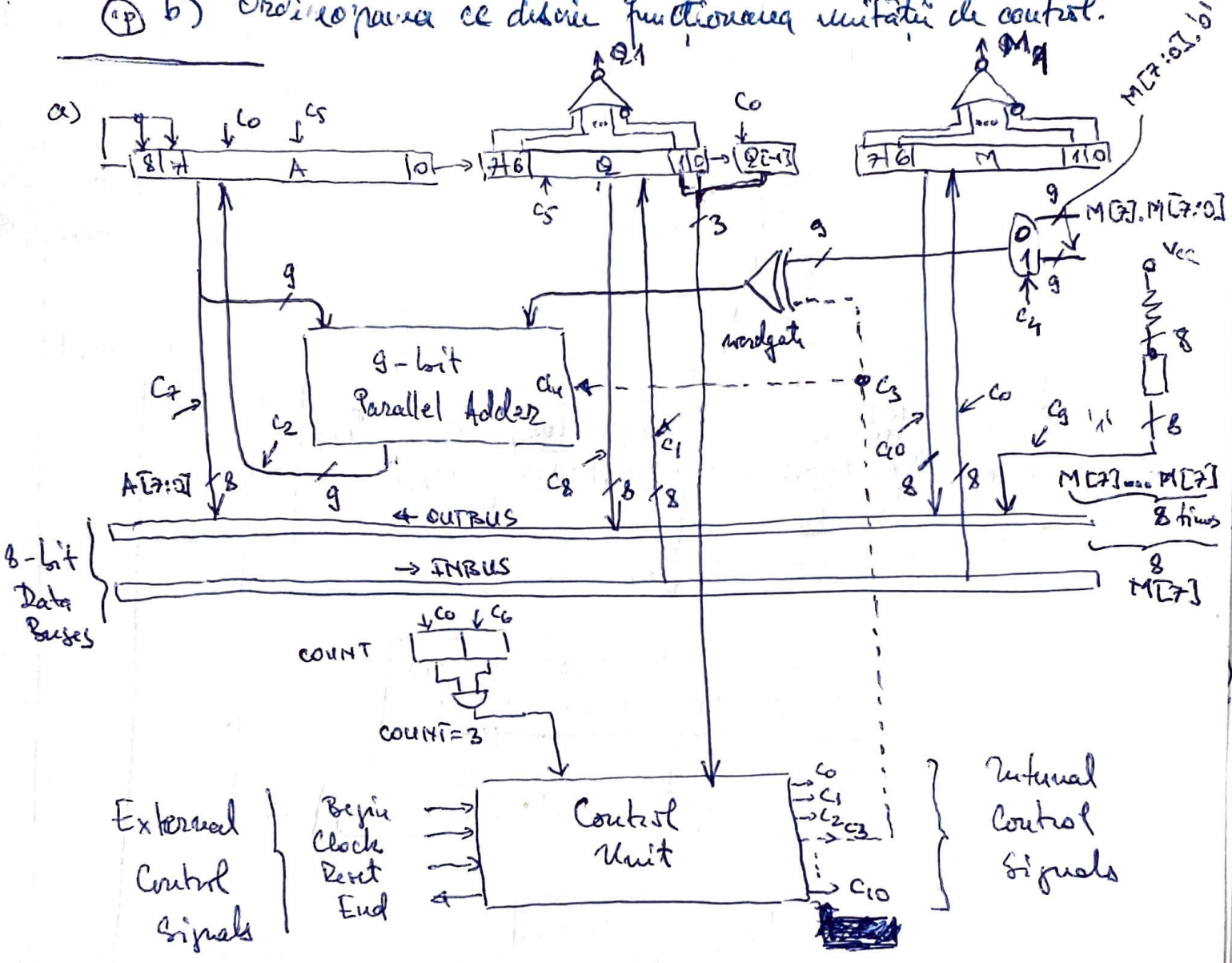
Rp (1) Să se completeze descrierea unui înmulțitor Booth radix 4, astfel încât să trateze cazul special al înmulțirii cu 1 (unul dintre operanzi este egal cu 1). Se cer:

(1p) a) Schema bloc hardware + semnalele de control

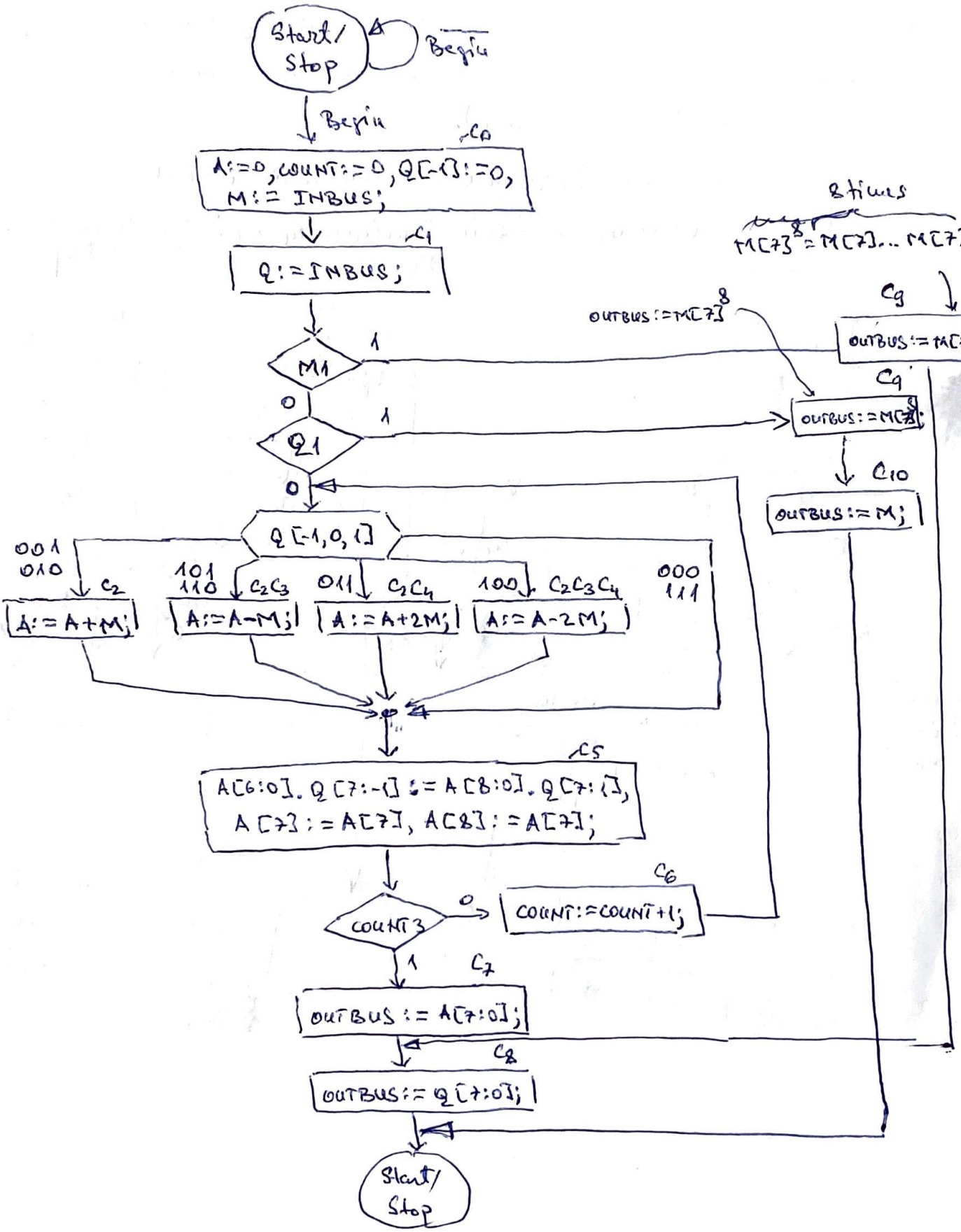
(1p) b) Ordinograma ce descrie funcționarea unității de control.

---

a)

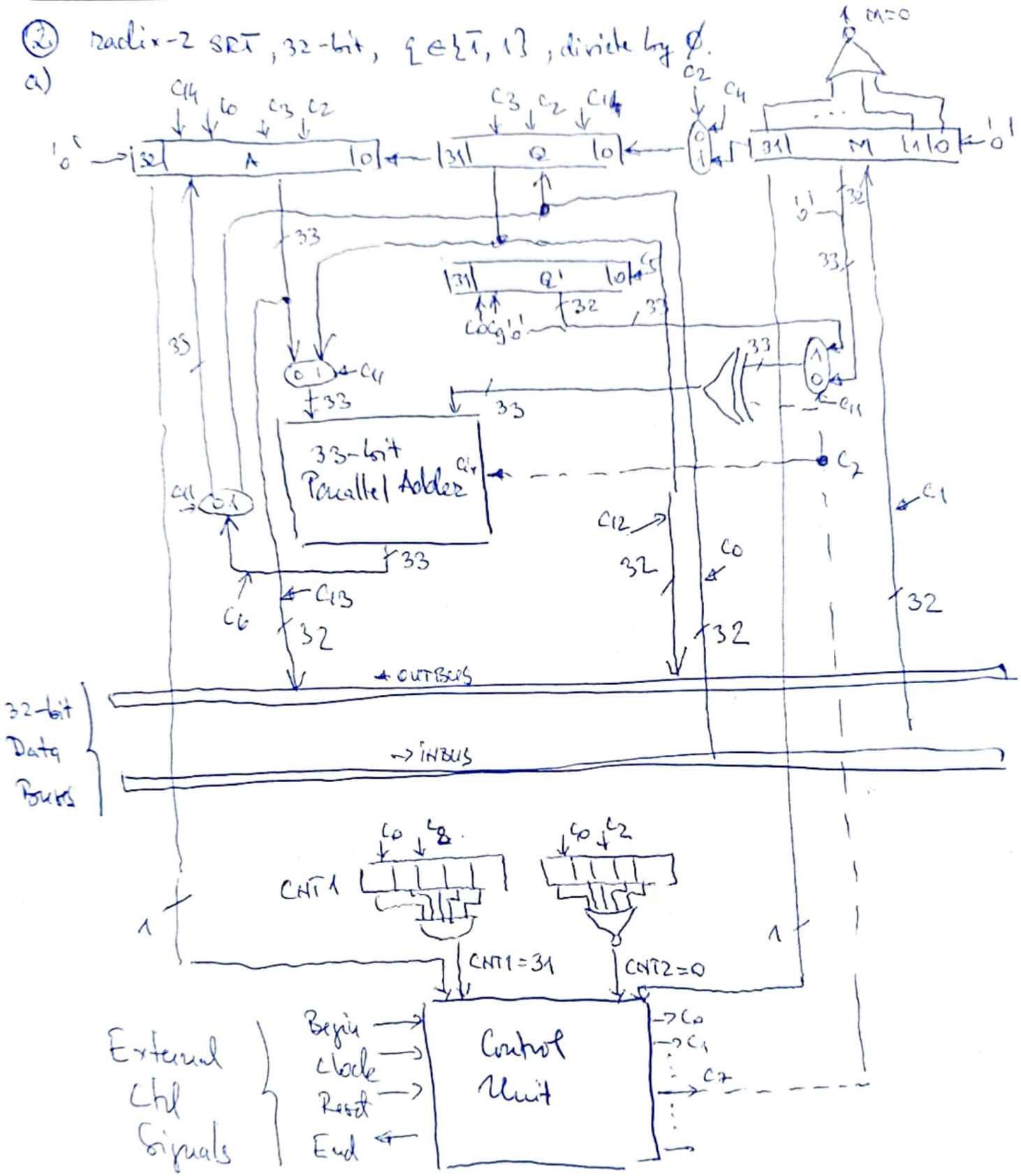


b)


Start/ Stop
Begin
Begin
C0
A:=0, COUNT:=0, Q[-1]:=0, M:=INBUS;
C1
Q:=INBUS;
M1
Q1
Q[-1,0,1]
001 010
C2
A:=A+M;
101 110
C2C3
A:=A-M;
011
C2C4
A:=A+2M;
100
C2C3C4
A:=A-2M;
000 111
C5
A[6:0], Q[7:-1]:=A[8:0], Q[7:1], A[7]:=A[7], A[8]:=A[7];
COUNT<3
C6
COUNT:=COUNT+1;
C7
OUTBUS:=A[7:0];
C8
OUTBUS:=Q[7:0];
Start/ Stop
8 times
M[7]^8 = M[7]...M[7]
OUTBUS:=M[7]^8
C9
OUTBUS:=M[7]^8;
C9'
OUTBUS:=M[7]^8;
C10
OUTBUS:=M;

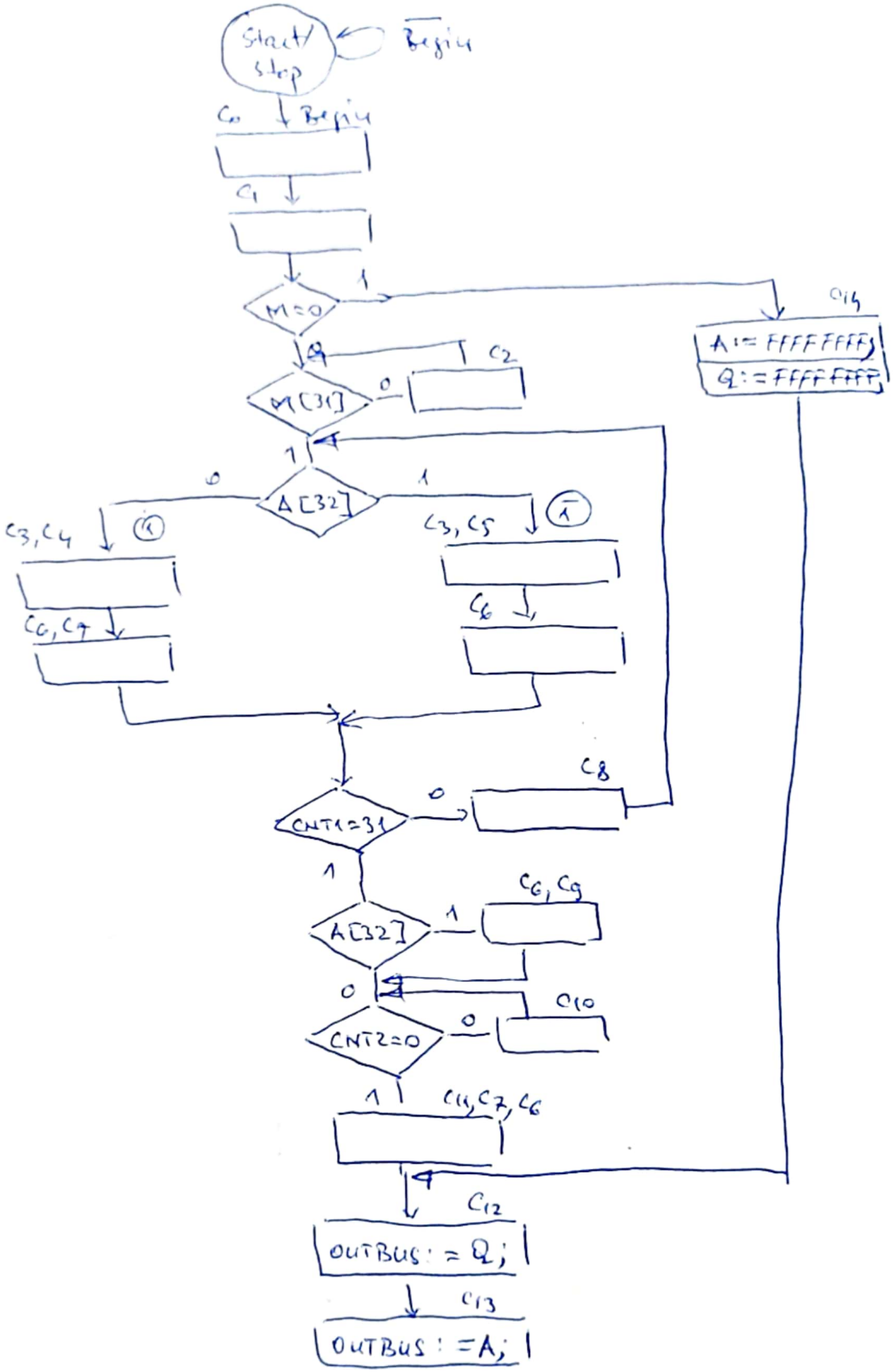
(2) radix-2 SRT, 32-bit, $q \in \{\bar{1}, 1\}$, divide by 0.

a)



Scanat cu CamScanner

2.b)



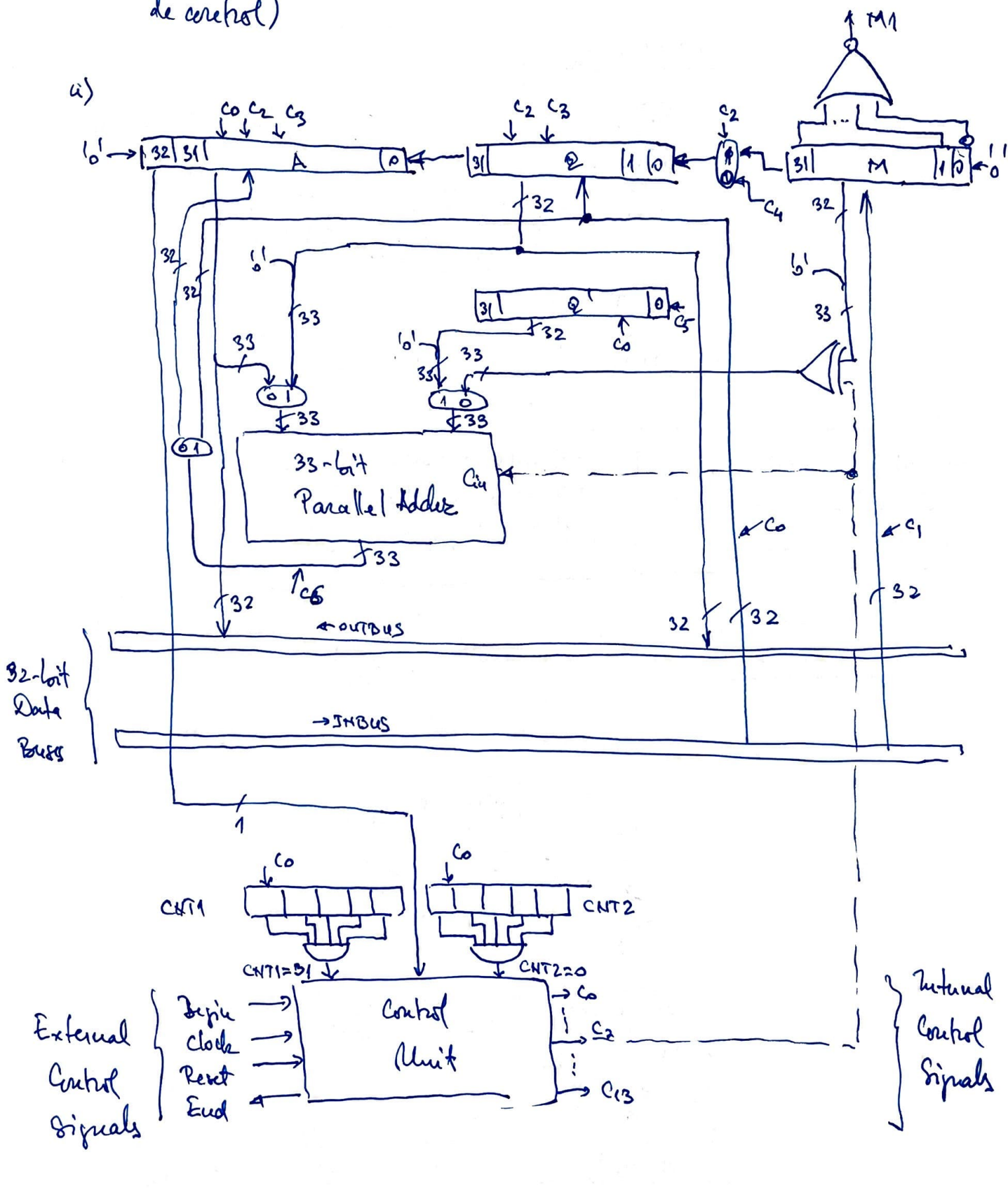
Scanat cu CamScanner

① Să se modifice dispozitivul de împărțire SRT radix-2 astfel încât acesta
② să împartă numere pe 32 de biți fără a genera cifra câtului 0 (doar 1 și $\bar{1}$) și să trateze în mod distinct situația de împărțire la 1. Se cer:

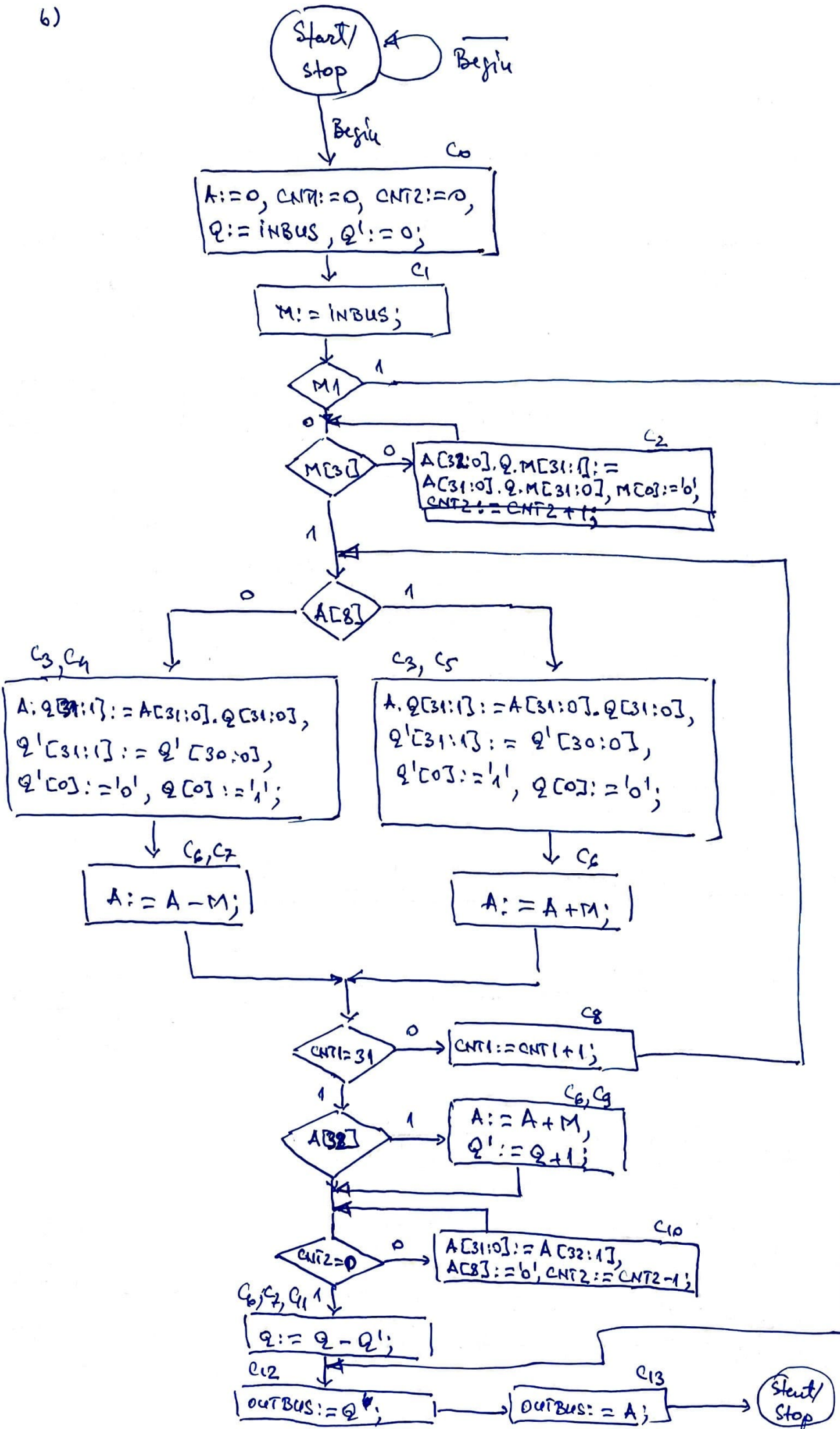
a) Schema bloc hardware folosind platforma Hayes + semnalele de control

b) Ordinograma ce descrie funcționarea unității de control (+ semnalele de control)

a)



6)

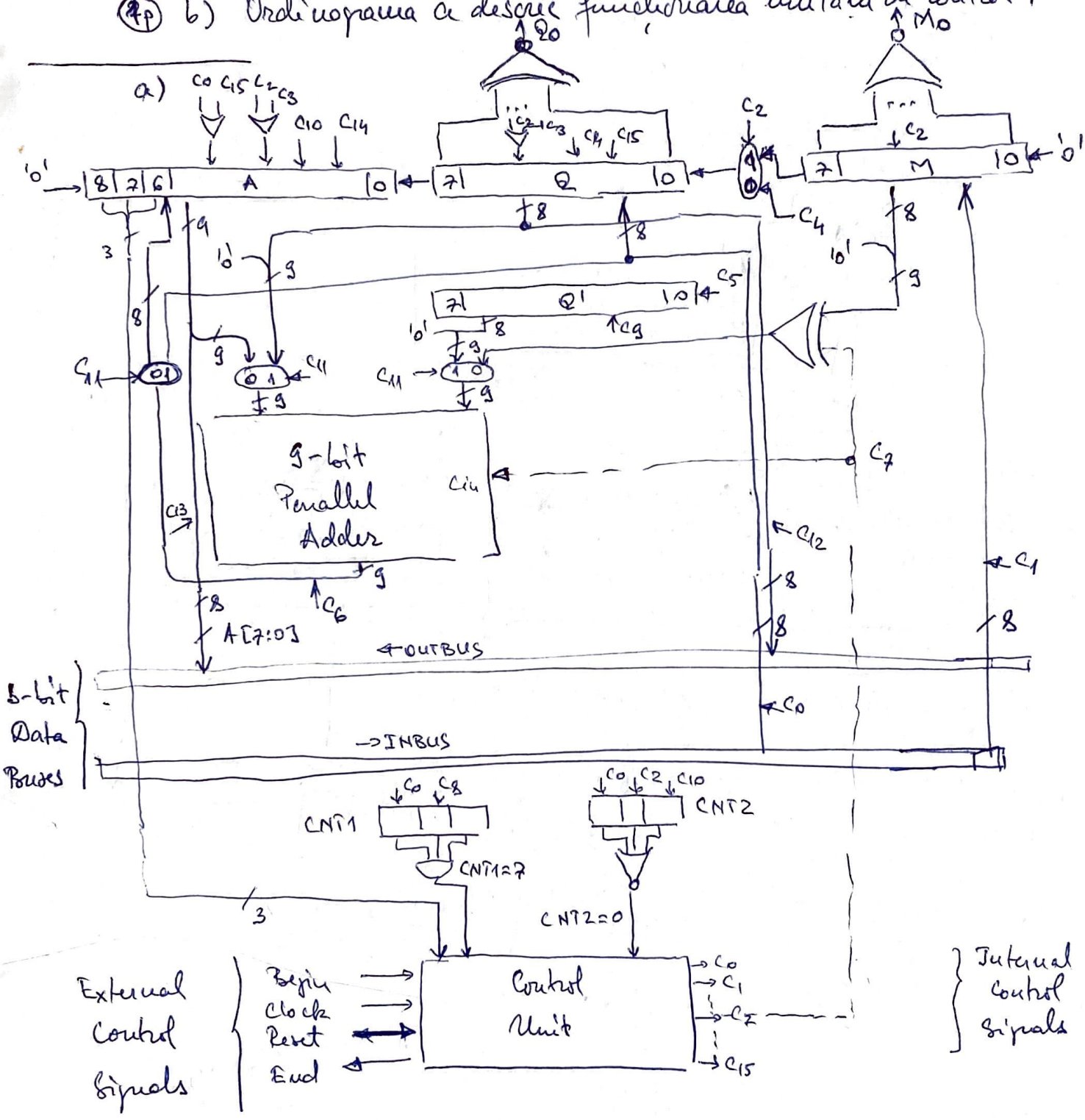

Start/ stop
Begin
Begin
C0
A:=0, CNT1:=0, CNT2:=0,
Q:=INBUS, Q1:=0;
C1
M:=INBUS;
M1
1
0
M[3]
0
C2
A[32:0], Q,M[31:1]:=
A[31:0], Q,M[31:0], M[0]:='0',
CNT2:=CNT2+1;
1
A[8]
0
1
C3, C4
A, Q[31:1]:=A[31:0], Q[31:0],
Q1[31:1]:=Q1[30:0],
Q1[0]:='0', Q[0]:='1';
C3, C5
A, Q[31:1]:=A[31:0], Q[31:0],
Q1[31:1]:=Q1[30:0],
Q1[0]:='1', Q[0]:='0';
C6, C7
A:=A-M;
C6
A:=A+M;
CNT1=31
0
C8
CNT1:=CNT1+1;
1
A[32]
1
C6, C9
A:=A+M,
Q1:=Q+1;
CNT2=0
0
C10
A[31:0]:=A[32:1],
A[8]:='0', CNT2:=CNT2-1;
C6, C7, C11
1
Q:=Q-Q1;
C12
OUTBUS:=Q;
C13
OUTBUS:=A;
Start/ Stop

A2
B1

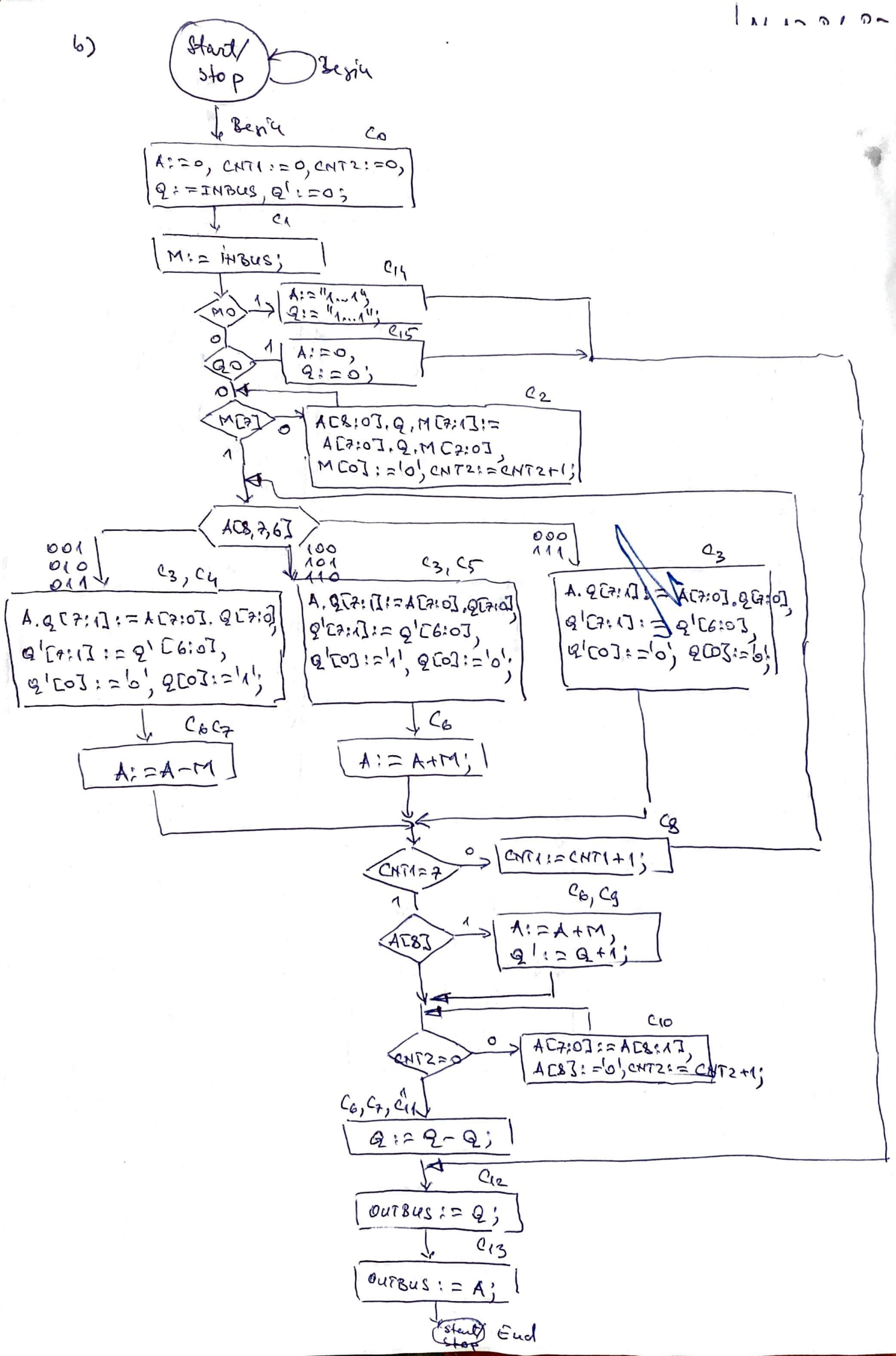
2p (1) Să se completeze descrierea unui împărțitor SRT radix 2, astfel încât să se trateze cazurile speciale de împărțire la zero (când atât câtul cât și restul sunt "1...1") și împărțirea a lui zero. Se cer:

(1p) a) Schema bloc hardware + semnale de control.

(4p) b) Ordinograma ce descrie funcționarea unității de control.



b)


Start/ stop
Beríš
Beríš
C0
A:=0, CNT1:=0, CNT2:=0,
Q:=INBUS, Q':=0;
C1
M:=INBUS;
C14
M0
1
A:="1...1",
Q:="1...1";
C15
0
Q0
1
A:=0,
Q:=0;
0
C2
M[7]
0
A[8:0], Q, M[7:1]:=
A[7:0], Q, M[2:0],
M[0]:='0', CNT2:=CNT2+1;
1
A[8,7,6]
001
010
011
C3, C4
A,Q[7:1]:=A[7:0], Q[7:0],
Q'[7:1]:=Q'[6:0],
Q'[0]:='0', Q[0]:='1';
100
101
110
C3, C5
A,Q[7:1]:=A[7:0], Q[7:0],
Q'[7:1]:=Q'[6:0],
Q'[0]:='1', Q[0]:='0';
000
111
C3
A,Q[7:1]:=A[7:0], Q[7:0],
Q'[7:1]:=Q'[6:0],
Q'[0]:='0', Q[0]:='0';
C6 C7
A:=A-M
C6
A:=A+M;
C8
CNT1=7
0
CNT1:=CNT1+1;
1
C6, C9
A[8]
1
A:=A+M,
Q':=Q+1;
C10
CNT2=0
0
A[7:0]:=A[8:1],
A[8]:='0', CNT2:=CNT2+1;
C6, C7, C11
Q:=Q-Q;
C12
OUTBUS:=Q;
C13
OUTBUS:=A;
Stavil stop
End

1.

4. Se dă un sistem de memorie, unde dimensiunea memoriei principale este de 32 GiB, adresarea este la nivel de byte, blocul este de 32 cuvinte, iar cuvântul este pe 32 biți. La ambele rânduri, mapearea este asociativă pe 4 căi. Dimensiunea de date a cache-ului este de 64 KiB.
a) Desenați schema logică a cache-ului, în maniera prezentată la curs, cu etichetarea lățimii semnalelor.
b) Calculați indexul din cache la care se mapează adresa: $04AAF8130_{hex}$.

a) $MM = 32\ GiB = 2^5 \cdot 2^{30} = 2^{35}\ B \Rightarrow p.a. = 35$

$1\ block = 32\ words = 2^5\ words \Rightarrow B.O. = 5$

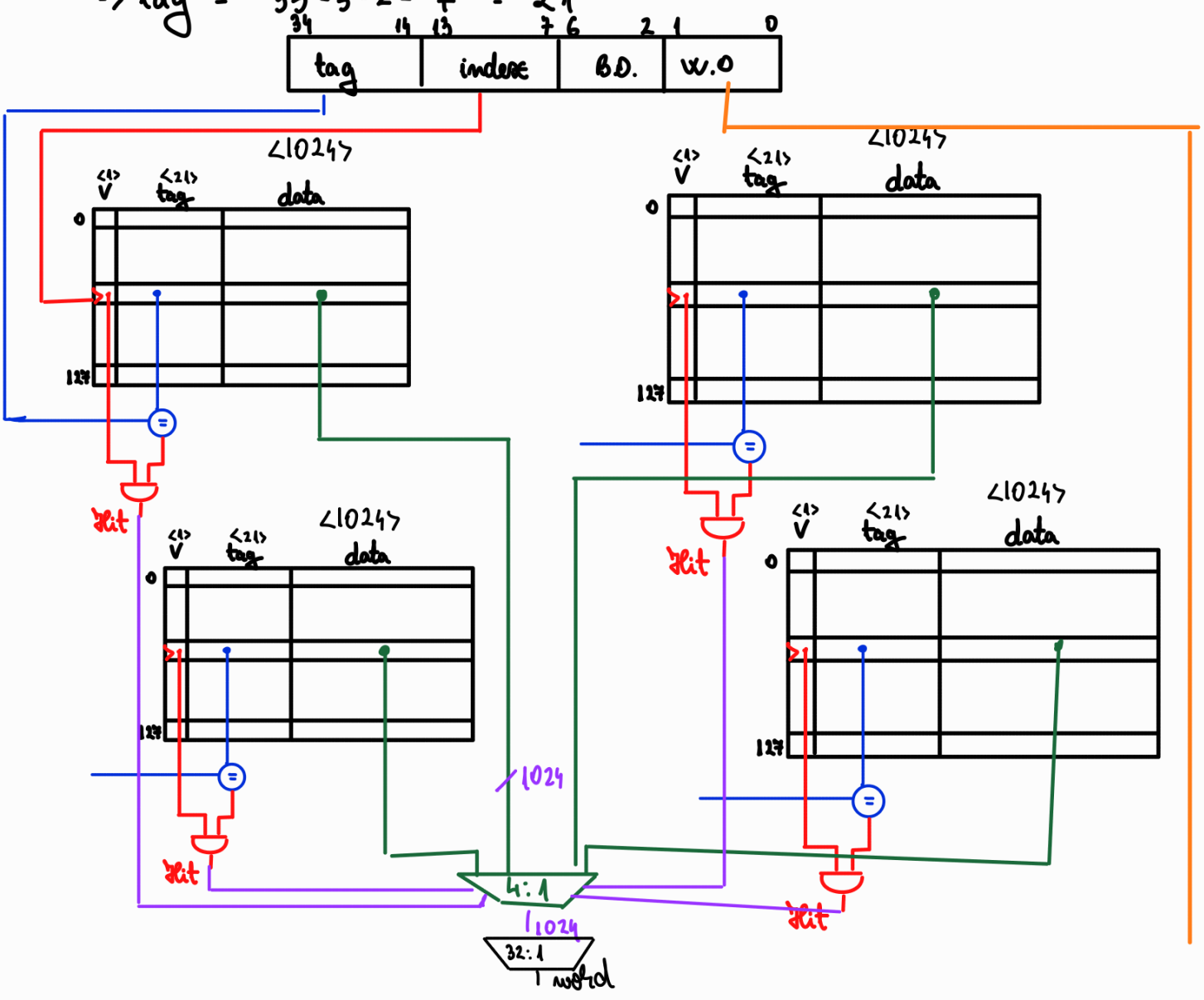
$1\ word = 32\ b = 4\ B = 2^2\ B \Rightarrow w.o = 2$

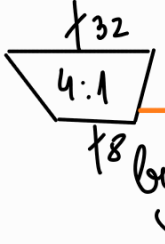
4 WAY SA

$data\ cache\ size = 64\ KiB = 2^6 \cdot 2^{10} = 2^{16}\ B$

$\frac{2^{16}}{2^5 \cdot 2^2 \cdot 2^2} = 2^7 \Rightarrow index = 7$

$\Rightarrow tag = 35 - 5 - 2 - 7 = 21$

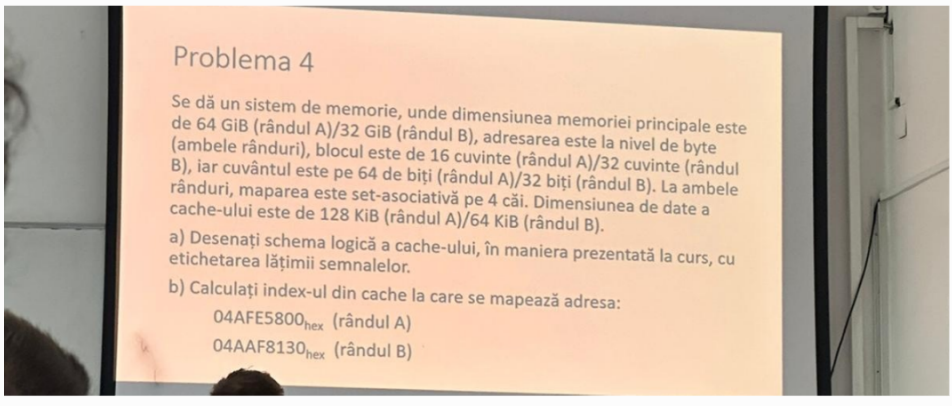




b) 0.4A.AF.8 1.30

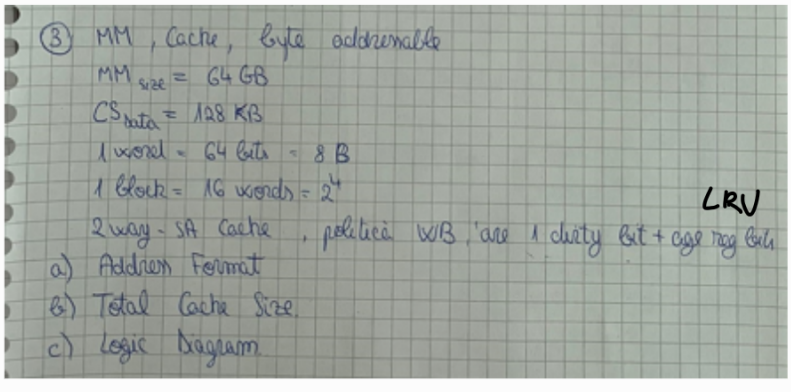
	tag <21>	index <7>	B.O. <4>	W.O <2>	line
0x04AAF.100	0.0001.00		11.00	00	4

Ex 2



la felca sus

Ex 3



a) MM = 64 GB = $2^6 \cdot 2^{30} = 2^{36}$ => p.a = 36

CS = 128 KiB = $2^7 \cdot 2^{10} = 2^{17}$

1 word = 64 bits = 8B = 2^3 B => W.O. = 3 b

1 block = 16 words = 2^4 words => B.O. = 4 b

} => $\frac{2^{17}}{2^3 \cdot 2^4 \cdot 2^1} = 2^{10}$ => index = 10

tag = 36 - 10 - 3 - 4 = 19

b) 1b dirty, 1b age (2 way)

total cache size = $2^{10} \cdot 2 \cdot (1b + 1b + 1b + 2^3 \cdot 2^4 \cdot 2^3 + 19b) =$

(linii, banks, V, dirty, age, data, tag)

$= 2^{11}(2^{10} + 22b) = 2^{11} \cdot 2^{10} + 2^{11} \cdot 2^2 = 2 \cdot 2^{20} + 2^3 \cdot 2^{10}$ - 2048 KiB + 8 KiB = 2056 KiB

Ex 4.

(3) (2p) Avem un computer load/store cu un sistem cache ce se caracterizează prin următoarele valori empirice atunci când rulăm un anumit program: 10% Data Miss Rate, 8% Instruction Miss Rate și 20% instrucțiuni load/store. În orice moment, 20% din blocuri sunt modificate (i.e., dirty). Programul determină un CPI de 2 clock cycles dacă cache-ul este perfect și constă din 10 000 de instrucțiuni în cod mașină. Calculatorul are o frecvență a clock-ului de 4.5 GHz. Penalitatea de miss e determinată de faptul că un access la BUS ~~înseamnă~~ necesită 10 cicluri de clock, lățimea BUS-ului este de 4 cuvinte, block-ul are 16 cuvinte iar cache-ul implementează o politică Write Back cu Write Allocate în cazul unui miss. Calculați:

(1p) a) AMAT, știind că $t_{ac} = 2$ cc.

(1p) b) CPU time.

a) Read Miss Penalty = $\underbrace{\frac{16 \text{ words}}{4 \text{ words}} \cdot 0,2 \text{ Bus writes}}_{\text{update}} + \underbrace{\frac{16 \text{ words}}{4 \text{ words}} \text{ Bus Reads}}_{\text{allocate}} = 4 \cdot 0,2 + 4 = 4 \cdot 1,2 = 4,8$ Bus accesses

$\Rightarrow$ Miss Penalty $= 4,8 \cdot 10 = 48$ C.C.

$CCT = \frac{1}{4,5 \cdot 10^{-9}} = 0,2222 \mu s$

Misses Per Instruction = Miss Rate · Mem. accesses per instruction ($1 + 0,2$)

alt. $\underset{MPi}{}$ $= 1 \times IMR + 0,2 \cdot DMR = 0,08 + 0,2 \cdot 0,1 = 0,1$

Miss Rate $= \frac{0,1}{1,2} = 0,0833$

$AMAT = \underset{CCT \cdot t_{ac}}{0,4444 \mu s} + \underset{MR \cdot MP \cdot CCT}{0,833 \cdot 48CC \cdot 0,2222} = 1,3328 \mu s$

b) $CPU_{TIME} = IC \cdot (CPI_{ideal} + MPi \cdot MP) \cdot CCT =$

$= 10\,000 (2 + 0,1 \cdot 48) \cdot 0,2222 = 15,1096 \mu s$

$AMAT = 0,4444 \mu s + 0,833 \cdot 48CC \cdot 0,222$

$P \quad CCT$

Ex 5

(3) Avem un computer L/S cu un sistem cache cu următoarele caracteristici:
(4p) prin măsurătorile valori empirice atunci când rulăm un anumit program: 9% Data Miss Rate, 8% Instruction Miss Rate și 10% instrucțiuni L/S. În orice moment 10% din blocuri sunt modificate (i.e., dirty). Programul determină un CPI de 3 c.c. dacă cache-ul este perfect și conține doar 1000 de instrucțiuni în cod mașină. Calculatorul are o frecvență a clock-ului de 5 GHz. Penalitatea de miss e determinată de faptul că un access la BUS necesită 8 cicluri de clock, lățimea BUS-ului este de 2 cuvinte, block-ul are 32 de cuvinte iar cache-ul implementează o politică Write Through cu Write No Allocate în cazul unui miss.
→ 10% ~~sunt~~ din acesse sunt scrieri, restul sunt citiri.

WT + WNA

$IC = 1000$

$MR = 9\% = 0,09$

$CR = 5\,GHz \Rightarrow CCT = \frac{1}{5 \cdot 10^9} = 0,2\,ns$

$IMR = 8\% = 0,08$

8 CC → 1 Bus acc.

L/S → 10% = 0,1

BUS width = 2 words

10% = 0,1 dirty

block size = 32 words

$CPI_{ideal} = 3\,CC$

10% scrieri

$t_{ac} = 1\,CC$

$$RMP_{WT} = \frac{\text{block size}}{\text{BUS width}} = \frac{32}{2} = 16 \text{ Bus Reads}$$

$WMP_{WT} = 1$

$$\text{Miss Penalty} = (0,1 \cdot 1 + 0,9 \cdot 16)\,8 = 14,5 \cdot 8 = 116 \text{ cc.}$$

$$\text{Misses/instr} = MR \cdot \text{Mem. acc/instr}^{1+0,1} =$$

$$= 1 \cdot 0,08 + 0,1 \cdot 0,09$$

$$\Rightarrow MR = \frac{0,089}{1,1} = 0,081$$

$$AMAT = (1 + MR \cdot MP^{\to CC}) \cdot CCT = (1 + 0,081 \cdot 116) \cdot 0,2 = 2,079\,ns$$

b) $$CPU_{time} = ic \cdot (CPI_{ideal} + MR \cdot MP^{\to CC} \cdot \text{Mem acc/instr}) \cdot CCT =$$
$$= 1000(3 + 0,081 \cdot 116 \cdot 1,1) \cdot 0,2 = 2667\,ns$$

Exo 6

(5) 0.2 L/S instr.
Clock rate = 3.6 GHz
CPI ideal = 3 c.c.
IMR = 8%
DMR = 5%
Miss penalty = 150 ns = $\lceil \frac{150\,ns}{0.2778} \rceil$ = 542 c.c.

Miss rate = 6%
Acc = 2 c.c.
optimisation →

a) $CPU\ time_{orig.}$ = IC × (CPI ideal + Misses per instr. × Miss Penalty) × Clock cycle time original

Clock cycle time = $\frac{1}{3.6 \times 10^9 s^{-1}}$ = $\frac{1}{3.6}$ ns = ~~0.277~~ 0.2778 ns

Misses per instr original = 0.08 + 0.2 × 0.05 = 0.09
Memory accesses per instr = 1.2

CPU time orig = IC × (3 + 0.09 × 540) × 0.277 ns
= IC × 14.2932 ns

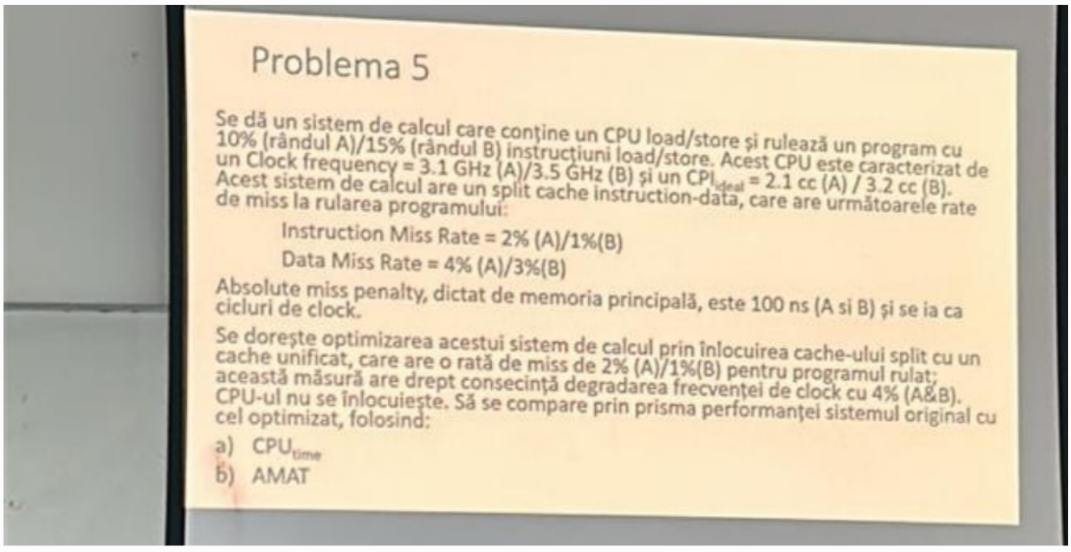
CPU time optim = IC × (CPI ideal + Mem acc per instr × Miss rate × Miss Penalty) × CCT

= IC (3 + 1.2 × 0.06 × 542) × 0.277 ns
= IC × 11.640 ns ✓ => CPU time optim mai bun!

b) $AMAT_{orig}$ = 0.277 ns + $\frac{0.09}{1.2}$ × 542 × 0.277 ns
= 11.537 ns

AMAT optim = 2 × 0.277 ns + 0.06 × 542 × 0.277 ns
= 9.562 ns ✓ AMAT optim mai bun!

6.



Problema 5

Se dă un sistem de calcul care conține un CPU load/store și rulează un program cu 10% (rândul A)/15% (rândul B) instrucțiuni load/store. Acest CPU este caracterizat de un Clock frequency = 3.1 GHz (A)/3.5 GHz (B) și un CPI_{ideal} = 2.1 cc (A) / 3.2 cc (B). Acest sistem de calcul are un split cache instruction-data, care are următoarele rate de miss la rularea programului:

Instruction Miss Rate = 2% (A)/1%(B)

Data Miss Rate = 4% (A)/3%(B)

Absolute miss penalty, dictat de memoria principală, este 100 ns (A si B) și se ia ca cicluri de clock.

Se dorește optimizarea acestui sistem de calcul prin înlocuirea cache-ului split cu un cache unificat, care are o rată de miss de 2% (A)/1%(B) pentru programul rulat; această măsură are drept consecință degradarea frecvenței de clock cu 4% (A&B). CPU-ul nu se înlocuiește. Să se compare prin prisma performanței sistemul original cu cel optimizat, folosind:

a) CPU_{time}

b) AMAT

A. $IC_{L/S} = 10\% = 0,1$

$CR = 3,1\ GHz \Rightarrow CCT = \frac{1}{3,1\ GHz} = 0,32\ ns$

$CPI_{ideal} = 2,1\ cc$

$iMR = 2\% = 0,02$

$dMR = 4\% = 0,04$

$MP = 100\ ns = \lceil \frac{100}{0,32} \rceil = 313\ cc.$

$MR_{new} = 2\% = 0,02$

$\Rightarrow CR_{new} = \frac{98}{100} \cdot CR = 0,96 \cdot 3,1 = 2,98\ GHz$

$\Rightarrow CCT_{new} = \frac{1}{2,98\ GHz} = 0,34\ ns$

a) $CPU_{time} = IC \cdot (CPI_{ideal} + MR \cdot MP \cdot Mem\ acc/instr) \cdot CCT$ ($MP \rightarrow cc$)

$Misses/instr = MR \cdot Mem\ acc/instr$ ($Mem\ acc/instr \rightarrow 1 + 0,1$ L/S)

$= 1 \cdot 0,02 + 0,1 \cdot 0,04$

$\Rightarrow MR = \frac{0,024}{1,1} = 0,022$

$CPU_{time_1} = IC(2,1 + 0,022 \cdot 313 \cdot 1,1) \cdot 0,32 = IC \cdot 2,876\ ns$

$CPU_{time_2} = IC \cdot (2,1 + 0,02 \cdot MP_2 \cdot 1,1) \cdot 0,34 = IC \cdot 2,921\ ns$

$MP_2 = \lceil \frac{100}{0,34} \rceil = 295\ cc$

$\Rightarrow$ 1 is better

b) $AMAT = (1 + MP \cdot MR) \cdot CCT$

$AMAT_1 = (1 + 313 \cdot 0,022) \cdot 0,32 = 2,524$ ms

$AMAT_2 = (1 + 295 \cdot 0,02) \cdot 0,34 = 2,346$ ms

2 is better

Ex 1

4. Mem virtuală 4TB
Mem fizică 256 GB
byte addressable
word = 64 biți = 8 bytes
block = 16 words
page = 16 KiB
TLB = 64 lines, direct mapping
Cache - 1 256 KiB, write back și write allocate
4 way SA, LRU cu reg uzurită

a) formatele cuvintelor de adresă fizică și virtuală
b) dim totale ale TLB și cache
c) schema comună TLB și cache

VM = 4 TB = $2^2 \cdot 2^{40} = 2^{42}$ => V.M. adr = 42 b

PM = 256 GB = $2^8 \cdot 2^{30} = 2^{38}$ => P.M. adr = 38 b

page = 16 KiB = $2^4 \cdot 2^{10} = 2^{14}$ B => Page Offset (P.O.) = 14 b

1 word = 64 b = 8 B = 2^3 B => W.O. = 3 b

1 block = 16 words = 2^4 words => B.O. = 4 b

TLB = 64 lines = 2^6 lines => index V.M. = 6 b

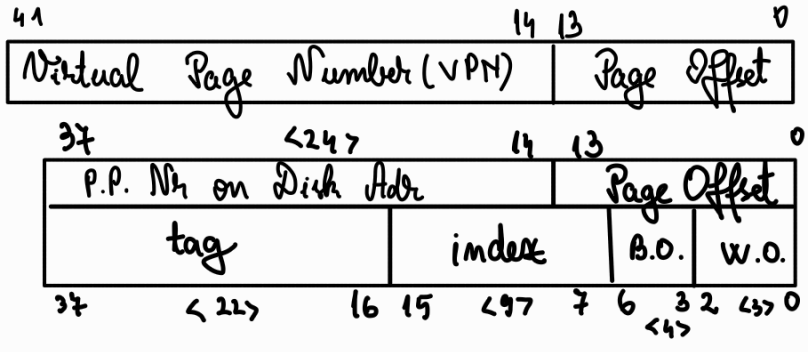
cache size = 256 KiB = $2^8 \cdot 2^{10} = 2^{18}$ B => $\frac{2^{18}}{2^3 \cdot 2^4 \cdot 2^2} = 2^9$ (blocks/bank) => index = 9 b

WB + WA

4 way

=> tag = 38 - 9 - 4 - 3 = 22 b

4 WAY - SA + LRU



b) Cache total size = $2^9 \cdot 2^2 \cdot (1\,b + 1\,b + 2\,b + 22\,b + 2^4 \cdot 2^3\,B) =$

index, 4way, v, dirty, age, tag

$= 2^{11} (26\,b + 2^7 B) = 2^{11} \cdot 2^2\,B + 2^{10} \cdot 2^8\,B = 2^{10}(8 + 256) =$

$= 264$ KiB

32 b : 8 = 4

$tag_{TLB} = 42 - 14 - 6 = 22b$

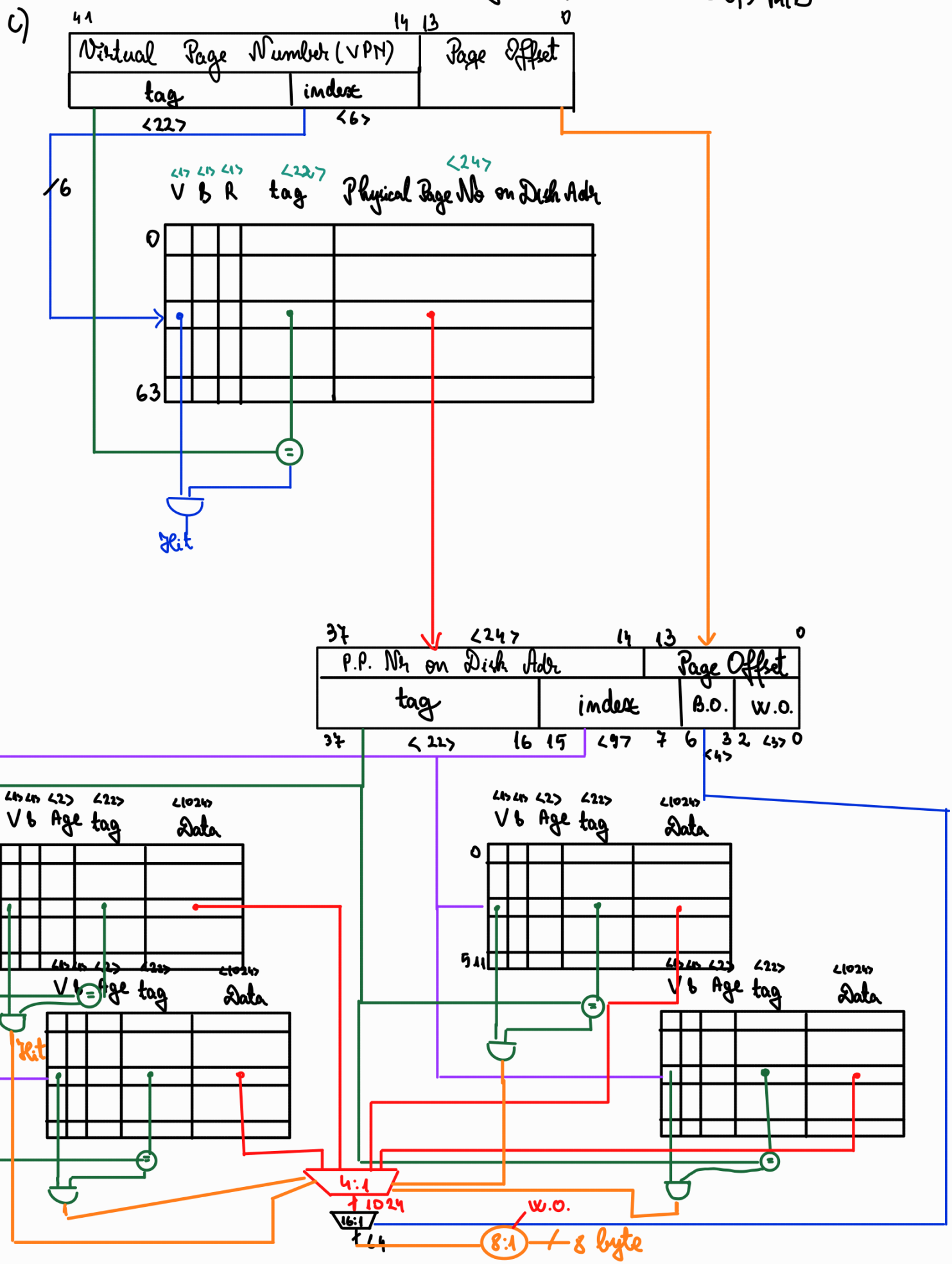
$64b = 2^3$

$TLB_{total\ size} = 2^6 \cdot (22b + 1b + 1b + 1b + 24b) = 2^6 \cdot 49b \sim 2^6 \cdot 2^3 \sim 2^9 B$

ind · tag · V · dirty · reference

$= 0{,}5\ KiB$

c)



Ex2 == Ex6

4) Virtual space = 64 TiB
Physical space = 128 GiB
byte-addressable
1 word = 64 biti = 8B
1 block = 8 words
1 page = 64 KiB
TLB [32 entries
full assoc.
Cache size data = 512 KiB
WT + WA
2-way SA
- random replacement

4a) Address format
↳ virtual
↳ physical

4b) TLB, Cache total size

4c) Schema bloc comun

a) VA $64TiB = 2^6 \cdot 2^{40} B = 2^{46} B$
PA $128GiB = 2^7 \cdot 2^{30} B = 2^{37} B$
Page $64KiB = 2^6 \cdot 2^{10} B = 2^{16} B$

46 − 16 = 30 ; 37 − 16 = 21

Virtual address

45 … 16	15 … 0
Virtual Page Number	Page offset
<30>	<16>

1 cache block = 2^3 words
1 word = 2^3 B

Physical address

36 … 15	15 … 0
Physical page no. or disk address	Page offset
<21>	<16>

BO - block offset
WO - word offset

37 − 18 = 19

For cache

36 … 17	16 … 6	5 4	3 2 1 0
Tag	Index	BO	WO
<19>	<12>	<3>	<3>

1 cache block = 2^3 words = $2^3 \cdot 2^3 B = 2^6$ B/block

512 KiB data cache = $2^9 \cdot 2^{10} B = 2^{19} B$ ⇒ $\frac{2^{19} B}{2^1 \text{ s.a.}} = 2^{18}$ B/bank

$\frac{2^{18} \text{ B/bank}}{2^6 \text{ B/block}} = 2^{12}$ blocks/bank → 12 = index size

b) Cache total size = $\underbrace{2}_{\text{s.a}} \times \underbrace{2^{12}}_{\text{blocks/banks}} (\underbrace{1\,\text{bit}}_{V} + \underbrace{19\,\text{bits}}_{\text{tag}} + \underbrace{2^6\,B}_{\text{data}}) =$

$= 2^{19}\,B$ data + $2^{12} \times 20$ bits $\cong 512\,\text{KiB} + 3 \cdot 2^{12}\,B$

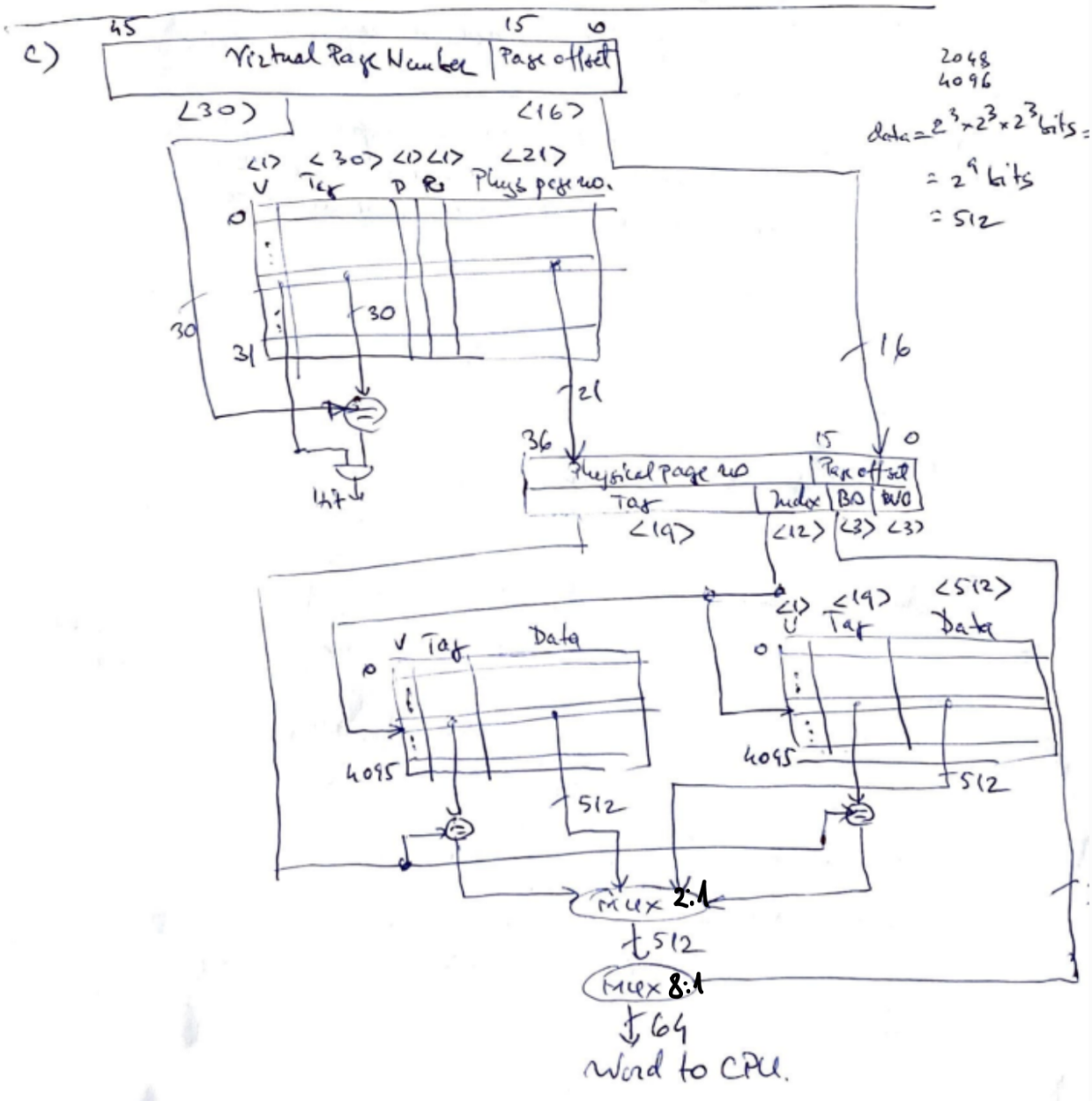
$= (512 + 12)\,\text{KiB} = 524\,\text{KiB}$

TLB total size = $2^5 \times$ (1 bit valid + 1 bit dirty + 1 bit acc +

+ 30 bits tag + 21 bits phys. addr) $\cong 2^5 \times 64$ bits =

$= 2^5 \times 2^3\,B = 2^8\,B = 0.25\,\text{KiB}$

c)



Ex 3

(4) (3p) Se dă un sistem de memorie, cu spațiul virtual de dimensiune 256 TiB, spațiu fizic de 256 GiB și adresare la nivel de byte. Cuvântul este pe 64 de biți, block-ul are 64 de cuvinte. O pagină de memorie are dimensiunea de 64 KiB, iar dimensiunea de date a cache-ului (SA pe 2 căi) este de 256 KiB. Cache-ul este Write Back, cu Write Allocate în caz de Write Miss și are o politică de înlocuire Random. TLB-ul are 64 de intrări și folosește mapare directă. Se cer:

(1p) a) Formatul adresei de memorie virtuală respectiv fizică

(1p) b) Dimensiunea totală a memoriei cache și a TLB-ului.

(1p) c) Schema bloc a mapării adresă fizică - adresă virtuală ce conține memoria cache și TLB-ul (ca în curs).

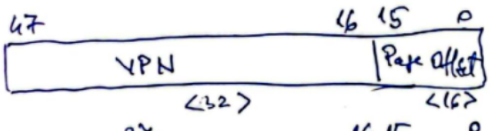
a) $VM_{size} = 2^8 \times 2^{40} B = 2^{48} B$ $\quad Page = 64 KiB = 2^{16} B$

$PM_{size} = 2^{38} B$

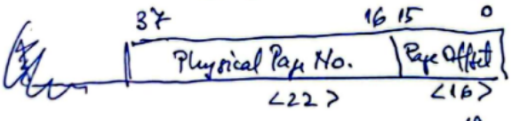
$1 word = 2^3 B$ → Word Offset

$1 block = 2^6 words = 2^6 \cdot 2^3 B = 2^9 B$

$TLB\ entries = 2^6$ → TLB Index



Virtual Memory

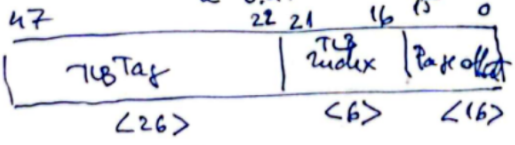


Physical Memory

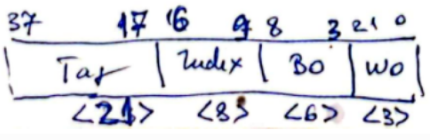
38 − 16 = 22

Cache data size $= 256 KiB = 2^{18} B = \frac{2^{18} B}{2^9 B/block} = 2^9$ blocks

Bank size $= \frac{2^9 \text{ blocks}}{2^1 \text{ S.a.}} = 2^8$ → Index blocks/bank.



Virt Mem



Phys. Mem

b) $CacheTSize = \underbrace{2^1}_{s.a} \times \underbrace{2^8}_{index} \times (\underbrace{2^9 B}_{data\,block} + \underbrace{16}_{V} + \underbrace{216}_{Tag}) = 2^9 \times 2^9 B + 2^2 \cdot 3B =$

16 dirty

$\cong 3B$

$= 2^{18}_{B} + 3 \times 0{,}5\,KiB$

$= 256\,KiB + 1.5\,KiB$

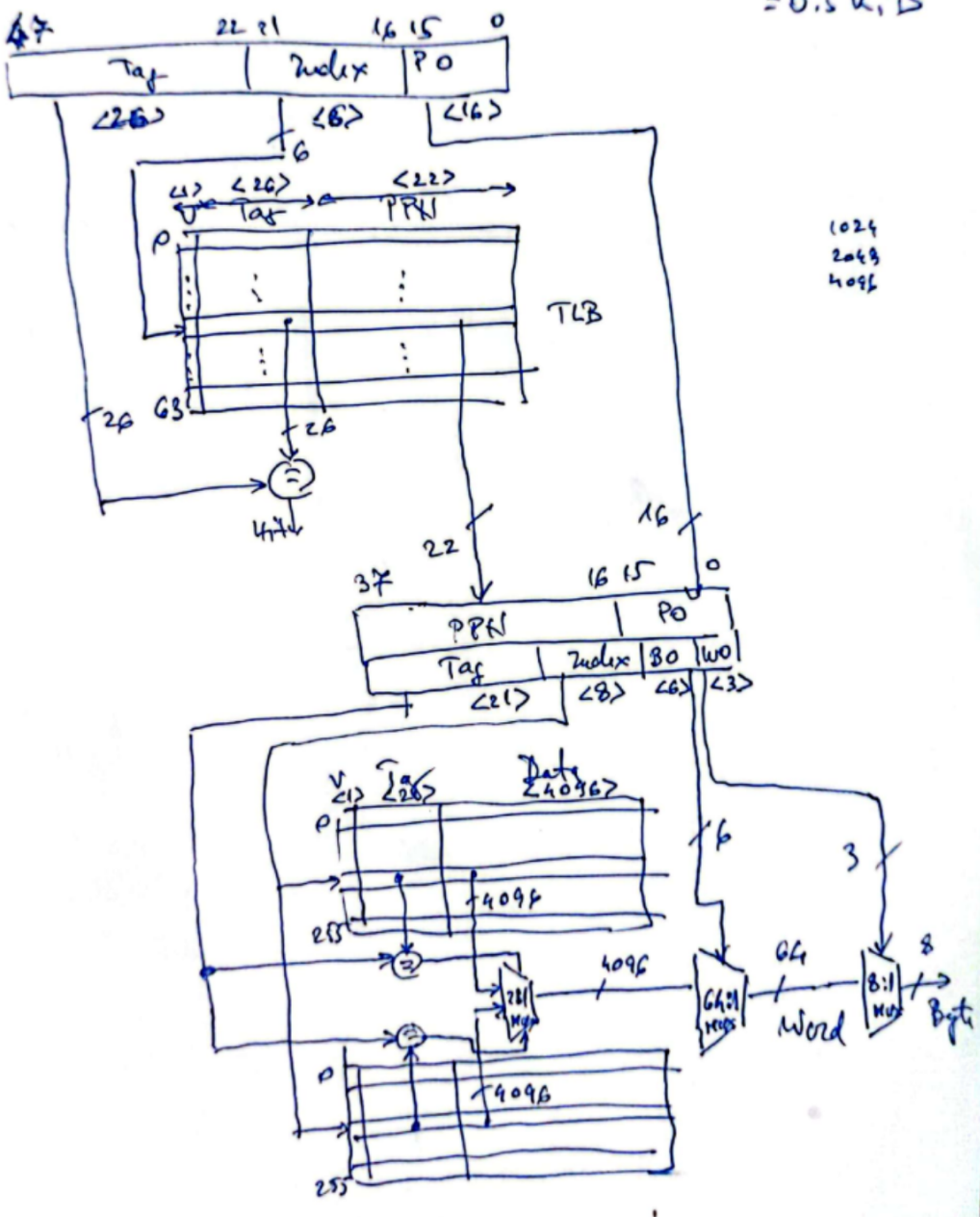
$= 257{,}5\,KiB$

$TLBTSize = 2^6 \times (\underbrace{1}_{V} + \underbrace{26}_{TLBtag} + \underbrace{22}_{PPN})\,bits = 2^6 \times \underbrace{49b}_{} \cong 2^6 \times 2^3 B$

$\cong 2^3 B = 2^9 B$

$= 0.5\,KiB$

c)



Ex 4. (4) (3p) Se dă un sistem de memorie cu spațiul virtual de dimensiune II
512 TiB, spațiu fizic de 128 GiB și adresare la nivel de octet.
Cuvântul este pe 64 de biți, blocul are 32 de cuvinte. O pagină de memorie are dimensiunea de 16 KiB, iar dimensiunea de date a cache-ului (4 căi) este de 128 KiB. Cache-ul este Write Through cu Write No Allocate în caz de Write Miss și are o politică de înlocuire de tip LRU cu age register. TLB-ul are 128 de intrări și folosește mapare set-asociativă pe 2 căi. Se cer:

(1p) b) Dimensiunea totală a memoriei cache și a TLB-ului
(1p) a) Formatul adresei de memorie virtuală, respectiv fizică
(1p) c) Schema bloc a mapării adresă virtuală – adresă fizică a conținut memoria cache și TLB-ul (ca în curs pt Fast MATH.)

37- 49-
14 14
23 35

a) Virtual Space → 512TiB = 2^{49}B
Physical Space → 128GiB = 2^{37}B
1 word = 64 bits = 2^3B
1 block = 2^5 words = $2^5 \cdot 2^3$B = 2^8B

1 page = 2^{14}B
Cache data size = 2^{17}B =
= $\frac{2^{17}\text{B}}{2^8\text{B/block}}$ = 2^9 blocks
Banks size = $\frac{2^9 \text{ blocks}}{2^{2 \text{ s.a}}}$ = 2^7 blocks

Virtual address: 48 | Virtual Page Number <35> | Page offset <14> (14 13 0)

Physical address: 36 | Physical page no. <23> | Page offset <14> (14 13 0)

128 entries TLB = 2^7 entries
$\frac{2^7}{2^1 \text{ sa}} = 2^6$ indexes

Virtual address: 48 | TLB Tag <29> | Index TLB <6> | Page offset <14> (20 19 14 13 0)

Physical address: 36 | Tag <22> | Index <7> | BO <5> | WO <3> (15 14 8 7 3 2 1 0)

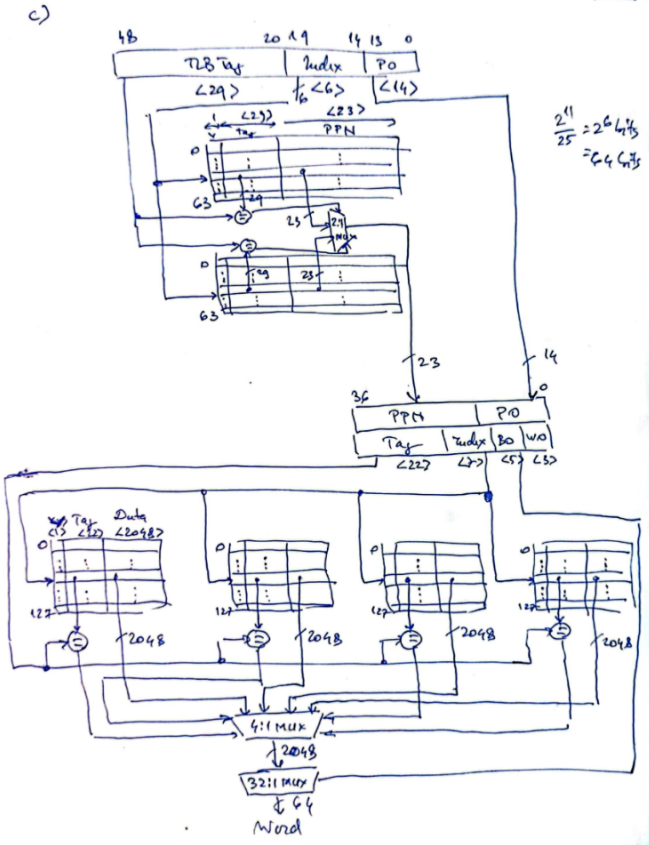
b) Cache total size = $\underbrace{2^2}_{\text{s.a.}} \times \underbrace{2^7}_{\text{index}} \times (\underbrace{2^8\text{B}}_{\text{data}} + \underbrace{1}_{\text{V}} + \underbrace{2}_{\text{age}} + \underbrace{22}_{\text{tag}}) \cong 4\text{B}$ =

... KiB = 130 KiB

TLB size = $2^7 \times (\underbrace{1}_{\text{V}} + \underbrace{29}_{\text{tag}} + \underbrace{23}_{\text{PPN}})$ ≅ 2^7 (24+32) bits =
= $2^7 \cdot 7$B ≅ $2^7 \cdot 8$B ≅ 1 KiB

with CamScanner

c)



Ex 5

1) 64 TiB – VM ⇒ $2^6 \cdot 2^{40} = 2^{46}$ B

a) 8 GiB – PM ⇒ $2^3 \cdot 2^{30} = 2^{33}$ B

byte addressing

16 KiB pages ⇒ $2^4 \cdot 2^{10} = 2^{14}$ B

32-entry TLB (2-way SA) → $\frac{2^5}{2} = 2^4$ ② index TLB

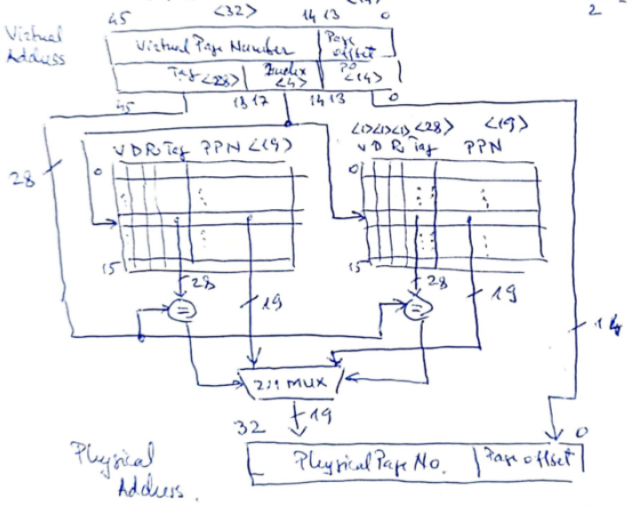
256 KiB Data Cache (2-way SA) = 2^{18} B = $\frac{2^{18}}{2^7} = 2^{11}$ blocks

1 block = 16 words = 2^4 w = $2^4 \cdot 2^3$ B = 2^7 B

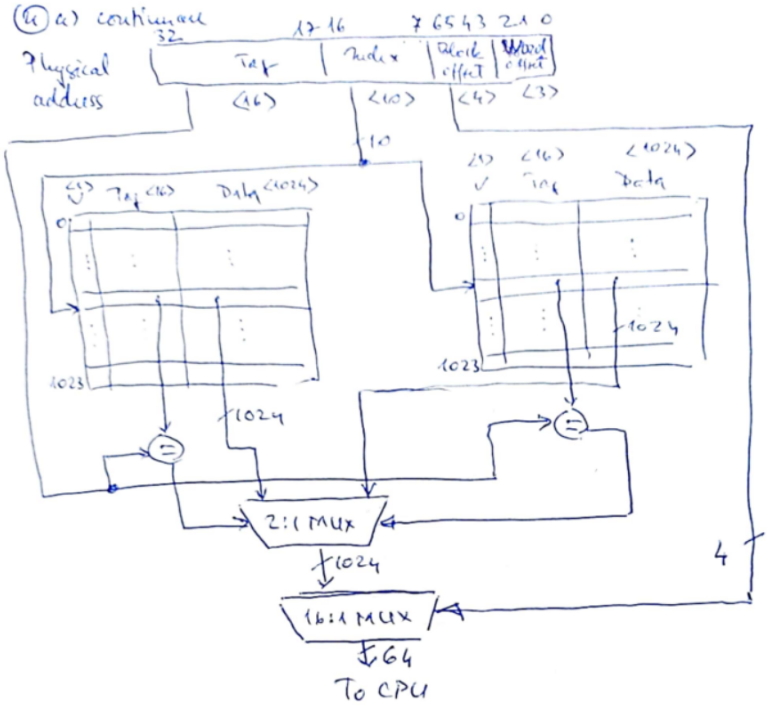
1 word = 64 bits = 2^6 bits = 2^3 B ⟨4⟩

$\frac{2^{11}}{2} = 2^{10}$ blocks / bank

32 − 17 = 15



② a) continuation



b) TLB size = $\underbrace{2^5}_{\text{total entries}} \times (\underbrace{31 \text{ bits}}_{\text{Tag, V, D, R}} + \underbrace{19 \text{ bits}}_{\text{PPN}}) = 2^5 \times 50 \text{ bits}$

$\cong 2^5 \times 2^6 \text{ bits}$

$= 2^5 \times 2^3 \text{ B} = 2^8 \text{ B}$

$= 0.25 \text{ KiB}$

Cache size total = $\underbrace{2}_{\text{s.a}} \times \underbrace{2^{10}}_{\text{index}} \times (\underbrace{17 \text{ bits}}_{\text{ctrl bits}} + 2^7 \text{ B})$

$= 2^{11} \times 17 \text{ bits} + 2^{18} \text{ B}$

$\cong \underbrace{2^{11} \times 2^5 \text{ bits}}_{2^{16} \text{ bits} = 2^{13} \text{ B} = 8 \text{ KiB}} + \underbrace{256 \text{ KiB}}_{\text{data}} \cong 264 \text{ KiB}$

Problema 6

Se dă un sistem de memorie cu următoarele caracteristici:

- Adrese virtuale pe
  - Rândul A: 45 de biți
  - Rândul B: 40 de biți
- Adrese fizice pe
  - Rândul A: 42 de biți
  - Rândul B: 32 de biți
- Dimensiunea paginii
  - A: 64 KiB
  - B: 16 KiB

Dimensiunea unei intrări în tabela de pagini este de 32 de biți (la ambele rânduri).

a) Prezentați formatul adreselor virtuale si fizice (page offset, VPN, PPN)

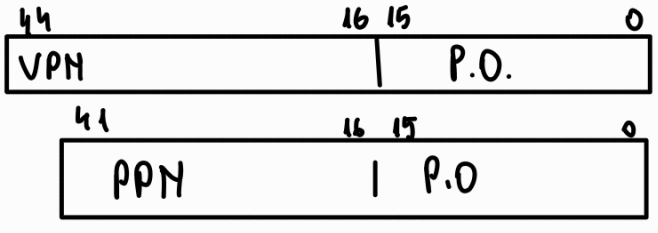
b) Pentru o mapare exhaustiva spațiu virtual -> spațiu fizic, care este dimensiunea tabelei de pagini?

a) (A) VM -> 45 b

PM -> 42 b

page size = 64 KiB $= 2^6 \cdot 2^{10} = 2^{16}$ => P.O. = 16 b

entrie = 32 b $= 2^2$ B



b) page table entries · table entrie = dim totală $= 2^{31}$ B

(2^2)

$$\frac{\text{adresă virtuală}}{\text{dim. pag}} \rightarrow \frac{2^{45}}{2^{16}} = 2^{29}$$

Formule

1. WB + WA

$$\text{Accesses via the BUS} = freq_r\left[\frac{words}{sec}\right] \cdot MR \cdot (\%\,write \cdot WMP^{b}_{WB} + \%\,citiri \cdot RMP^{b}_{WB})$$

$$\text{Write Miss Penalty}^{b}_{WB} = \text{Read Miss Penalty}^{b}_{WB}$$

$$WMP^{b}_{WB} = \frac{\text{block size}}{\text{BUS width}} \cdot \text{BUS Reads} + \frac{\text{block size}}{\text{BUS width}} \cdot \text{dirty} \cdot \text{BUS writes}$$

ALLOCATE — UPDATE

2. WB + WNA

$$\text{Accesses via the BUS} = freq_r\left[\frac{words}{sec}\right] \cdot MR \cdot (\%\,write \cdot WMP^{b}_{WB} + \%\,citiri \cdot RMP^{b}_{WB})$$

$$RMP^{b}_{WB} = \frac{\text{block size}}{\text{BUS width}} \cdot \text{BUS Reads} + \frac{\text{block size}}{\text{BUS width}} \cdot \text{dirty} \cdot \text{BUS writes}$$

ALLOCATE — UPDATE

$$WMP^{b}_{WB} = 1$$

3. WT + WA

$$\text{Accesses via the BUS} = freq_r\left[\frac{words}{sec}\right] \cdot MR \cdot (\%\,write \cdot WMP^{b}_{WT} + \%\,citiri \cdot RMP^{b}_{WT}) +$$

pt. Hit P. → $+ freq_r\,(1 - MR) \cdot WHP^{b}_{WT} \cdot \%\,write$

$$RMP^{b}_{WT} = \frac{\text{block size}}{\text{bus width}}\ BR$$

$$WMP^{b}_{WT} = \frac{\text{block size}}{\text{bus width}}\ BR + 1\ BW$$

$$WHP^{b}_{WT} = 1\ BW$$

4. WT + WNA

$$\text{Accesses via the BUS} = freq_r\left[\frac{words}{sec}\right] \cdot MR \cdot (\%\,write \cdot WMP^{b}_{WT} + \%\,citiri \cdot RMP^{b}_{WT}) +$$

pt. Hit P. → $+ freq_r\,(1 - MR) \cdot WHP^{b}_{WT} \cdot \%\,write$

$$RMP^{b}_{WT} = \frac{\text{block size}}{\text{bus width}}$$

$$WMP^{b}_{WT} = 1\ \text{BUS Write}$$

$$WHP^{b}_{WT} = 1\ \text{BUS Write}$$

Ex_1 Un sistem de calcul are un procesor ce trimite referințe către memoria cache la o frecvență de 10^8 words/second, unde 10% sunt scrieri, iar 90% citiri.

Cache-ul are un miss rate de 10% și un block de 8 cuvinte.

Bus-ul are un bandwidth de 10^9 accese/secundă și o lățime (bus bandwidth) de 1 cuvânt.

La dtca moment, 20% din blocuri sunt dirty.

Să se calculeze procentajul din bus bandwidth folosit din cauza imperfecțiunii cache-ului în urm. cazuri: A_2, B_1 → date diferite

A1 a) WB+WA b) WT + WNA

a)

$$\text{Accesses via the BUS} = \text{frecv}\left[\frac{\text{words}}{\text{sec}}\right] \cdot MR \cdot (\%\text{ scrieri} \cdot WMP^{b}_{WB} + \%\text{ citiri} \cdot RMP^{b}_{WB})$$

$$WMP^{b}_{WB} = \frac{\text{block size}}{\text{BUS width}} \cdot \text{BUS Reads} + \frac{\text{block size}}{\text{BUS width}} \cdot \text{dirty}^{WB} \cdot \text{BUS writes} =$$

ALLOCATE UPDATE

$$= \frac{8}{1} + 0{,}2 \cdot \frac{8}{1} = 1{,}2 \cdot 8 = 9{,}6 \text{ BUS Accesses}$$

$$AVTB = 10^8 \cdot 0{,}1 \cdot (0{,}1 \cdot 9{,}6 + 0{,}9 \cdot 9{,}6) = 10^7 \cdot 9{,}6 \text{ BUS Accesses}$$

$$\%\text{ Used BUS}_{WB+WA} = \frac{AVTB}{\text{BUS bandwith}} = \frac{10^7 \cdot 9{,}6}{10^9} = 9{,}6\%$$

b)

$$\text{Accesses via the BUS} = \text{frecv}\left[\frac{\text{words}}{\text{sec}}\right] \cdot MR \cdot (\%\text{ scrieri} \cdot WMP^{b}_{WT} + \%\text{ citiri} \cdot RMP^{b}_{WT}) +$$

pt. Hit P. → $+ \text{frecv} \cdot (1 - MR) \cdot WHP^{b}_{WT} \cdot \%\text{ scrieri}$

$$RMP^{b}_{WT} = \frac{\text{block size}}{\text{bus width}} = \frac{8}{1} = 8 \text{ BUS Reads}$$

$$WMP^{b}_{WT} = 1 \text{ BUS Write}$$

WHP^{b}_{WT} = 1 BUS Write

$$AVTB = 10^8 \cdot 0,1 \cdot (0,1 \cdot 1 + 0,9 \cdot 8) + 10^8 \cdot 0,9 \cdot 1 \cdot 0,1 =$$
$$= 10^7 \cdot (7,3 + 0,9) = 8,2 \cdot 10^7$$

$$\%\ \text{Used BUS}_{WT+WNA} = \frac{8,2 \cdot 10^7}{10^9} = 8,2\%$$

B_2 am schimbat : a) WB + WNA b) WT + WA

a)

$$\text{Accesses via the BUS} = \text{frecv}\left[\frac{\text{words}}{\text{sec}}\right] \cdot MR \cdot (\%\ \text{scrieri} \cdot WMP^{b}_{WB} + \%\ \text{citiri} \cdot RMP^{b}_{WB})$$

$$RMP^{b}_{WB} = \frac{\text{block size}}{\text{BUS width}} \cdot \text{dirty BUS Writes} =$$

$$WMP_{WB} = 1$$

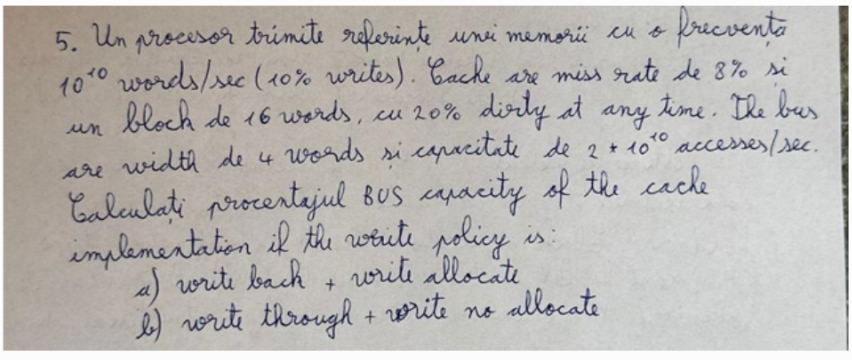
Nerezolvate

...rcise 5

A processor sends references to the memory with frequency 10^{10} words/sec (10% writes (A)/15% writes (B)). The cache has a miss rate of 8% (A)/ 12% (B) and a block of 16 words (A)/4 words (B), with 20% (A)/14%(B) being dirty at any time. The bus has a width of 4 words and a capacity of $2^* 10^{10}$ accesses/sec (A,B). Calculate the percentage of BUS capacity used because of the cache imperfection if the write policy is:

a) write back + write allocate

b) write through + write no allocate



5. Un procesor trimite referințe unei memorii cu o frecvență 10^{10} words/sec (10% writes). Cache are miss rate de 8% și un block de 16 words, cu 20% dirty at any time. The bus are width de 4 words și capacitate de $2 * 10^{10}$ accesses/sec. Calculați procentajul BUS capacity of the cache implementation if the write policy is:

a) write back + write allocate

b) write through + write no allocate

(1)
```
for( i=0, p=0, m=0; i<maxe; i++) {
    if(i%2==0){
        p++;
    }
    else  m++;
}
```

$r_0 \longrightarrow$ adr
$r_1 \longrightarrow$ i
$r_2 \longrightarrow$ p
$r_3 \longrightarrow$ m
$r_4 \longrightarrow$ maxe

```
init:   MOV   r1 , #0
        MOV   r2 , # 0
        MOV   r3 , # 0
        ABR   r0 , maxe
        LBR   r4 , [r0]

loop:   CMP  r1, r4        i- maxe
        BGE  end
        ANB  r5 , r1 , #1
        CMP  r5 , #0
        ABBEQ  r2 , r2 , #1  ⟶ p++
        ABBNE  r3, r3, #1
        ABB  r1, r1, #1
    B loop

end:  ABR  r0, p              ABR  r0, m
      STR  r2 , [r0]          STR  r3 , [r0]
```

(2.)
```
for ( i=0, m=0 ; i < maxe; i++) {
    m = m+i;
    if( m > 2*maxe)   m = 0;
}
```

$r_0 \rightarrow$ adr
$r_1 \rightarrow$ i
$r_2 \rightarrow$ m
$r_3 \rightarrow$ maxe

```
init:  MOV  r1, #0
       MOV  r2, #0
       ABR  r0, maxe
       LBR  r3, [r0]
loop:  CMP  r1, r3
       BGE  end
       Abb  r2, r2, r1
       Abb  r4, r3, r3
       CMP  r2, r4
       MOVGT r2, #0
       Abb  r1, r1, #1
       B loop
end:   ABR  r0, m
       STR  r2, [r0]
```

③.
```
for( i=1, s=0 ; i<=n ; i++) {
        if(i%3==0) s += i*i;
}
```

$r_0 \longrightarrow$ adr
$r_1 \longrightarrow$ i
$r_2 \longrightarrow$ s
$r_3 \longrightarrow$ n
$r_4 \longrightarrow$ tmp
$r_5 \longrightarrow$ rest

```
init:  MOV  r1, #1
       MOV  r2, #0
       MOV  r0, n
       MOV  r3, [r0]
       MOV  r5, #1
loop:  CMP  r1, r3
       BGT  end
       CMP  r5, #0
       BNE  skip
       MUL  r4, r1, r1
       ADD  r2, r2, r4        s+= i*i
skip:  ADD  r1, r1, #1
       ADD  r5, r5, #1
       CMP  r5, #3
       MOVEQ r5, #0
    B loop
end:   ADR  r0, s
       STR  r2, [r0]
```